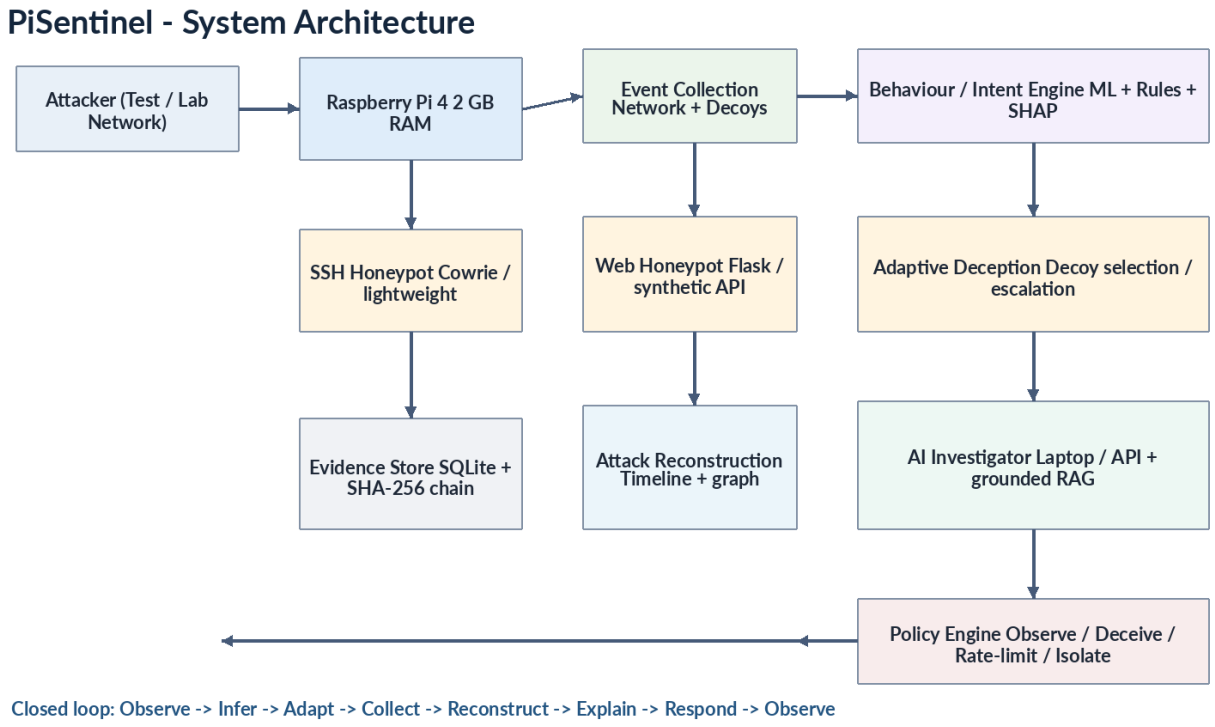
PiSentinel

Adaptive Cyber Deception and Explainable Autonomous Digital Forensics Platform for Edge Devices

A complete project 101: idea, architecture, approach, algorithms, implementation plan, experiments, risks, and final demonstration

OBSERVE | DECEIVE | UNDERSTAND | RESPOND



Scope anchor

This document describes the intended build around a Raspberry Pi 4 with 2 GB RAM. The Pi performs lightweight collection, decoy hosting, feature extraction, classification, on-demand SHAP, evidence chaining and basic adaptive control. Heavy model training, large-scale visualization and LLM investigation are offloaded to a trusted laptop/API.

Prepared as the master build reference.

1. Executive Overview

PiSentinel is a defensive cybersecurity research platform built around a resource-constrained edge device. Instead of only detecting an attacker, it creates a controlled fake environment, studies the attacker's behaviour, estimates intent, adapts the deception presented to the attacker, preserves tamper-evident evidence, reconstructs the attack sequence, and uses evidence-grounded AI to produce an investigation summary. The design is deliberately modular so the core system can run on a Raspberry Pi 4 while computationally expensive work is performed elsewhere.

One-line idea

Turn a Raspberry Pi into a small, explainable cyber-defense laboratory that does not merely block attacks - it learns from attacker behaviour, keeps the attacker engaged with controlled decoys, preserves what happened as evidence, and explains the incident.

1.1 What makes the project different

- Detection is only one layer. The system treats attacker interaction as a sequence of behaviours rather than a single alert.
- Deception is adaptive. The next fake asset depends on the current inferred behaviour and confidence.
- Explainability is behavioural. SHAP explains which session-level features pushed the model toward an intent/threat decision.
- Forensics is built into the system from the start. Events are timestamped, linked, hashed and reconstructed into a timeline/attack graph.
- The AI investigator is evidence-grounded. It receives verified structured evidence and is not given unrestricted shell/root access.
- The design is resource-aware. The Raspberry Pi 4 is treated as an edge sensor and control point, not as a machine-learning supercomputer.

1.2 Core research question

Research question

Can adaptive, intent-driven cyber deception combined with tamper-evident forensic reconstruction improve attacker engagement and post-incident understanding on a resource-constrained edge device?

1.3 Main contribution claims to test

Contribution	What will actually be implemented	How it can be evaluated
Adaptive deception	Rule-driven selection/escalation of fake services or content based on observed behaviour	Static vs adaptive comparison
Intent-aware defence	Behaviour classes and confidence drive the next defensive action	Intent classification metrics + decision traces
Explainable decisions	SHAP on aggregated session features	Feature attribution quality + analyst usefulness
Tamper-evident forensics	Hash-chained event records and integrity verification	Modification detection + evidence completeness
Attack reconstruction	Timeline and graph derived from ordered events	Stage/order reconstruction accuracy
Evidence-grounded AI	LLM creates reports only from verified JSON evidence	Groundedness / hallucination checks
Edge deployment	Core loop runs on Pi 4 2 GB	CPU, RAM, latency, storage, network

Contribution	What will actually be implemented	How it can be evaluated
		overhead

2. Table of Contents / Navigation

- 3. Problem and Motivation
- 4. System Idea in Plain English
- 5. Threat Model and Lab Boundaries
- 6. Final System Scope
- 7. Architecture
- 8. Module-by-Module Design
- 9. Data Model and Event Schema
- 10. Behaviour Features and Intent Classification
- 11. SHAP Explainability
- 12. Adaptive Cyber Deception
- 13. Digital Forensics and Evidence Integrity
- 14. Attack Timeline and Graph Reconstruction
- 15. Evidence-Grounded AI Investigator
- 16. Risk-Aware Response Engine
- 17. Implementation Stack
- 18. Raspberry Pi 4 Resource Strategy
- 19. Build Plan - Week by Week
- 20. Experimental Methodology
- 21. Metrics and Evaluation
- 22. Test Cases and Demo Scenarios
- 23. Security, Safety and Isolation
- 24. Failure Modes and Fallbacks
- 25. What Not to Build
- 26. Final Deliverables
- 27. Viva / Evaluation Questions
- 28. Glossary
- 29. Final Master Checklist

3. Problem and Motivation

Traditional defensive systems often answer a narrow question: “Is this activity malicious?” PiSentinel is designed around a broader operational question: “What is the attacker trying to do, how are they progressing, what evidence proves it, and what should the system do next?”

3.1 The practical gap

Layer	Typical focus	PiSentinel focus
Detection	Flag suspicious activity	Detect and characterize behaviour
Explanation	Why was this event classified?	Which features support the inferred intent?
Deception	Static honeypot / fake service	Adapt what is exposed based on attacker behaviour
Forensics	Logs stored separately	Evidence captured as part of the control loop
Investigation	Human analyst manually correlates logs	Structured timeline + attack graph + grounded AI summary
Response	Block or alert	Choose among observe, deceive, rate-limit and isolate using confidence/risk

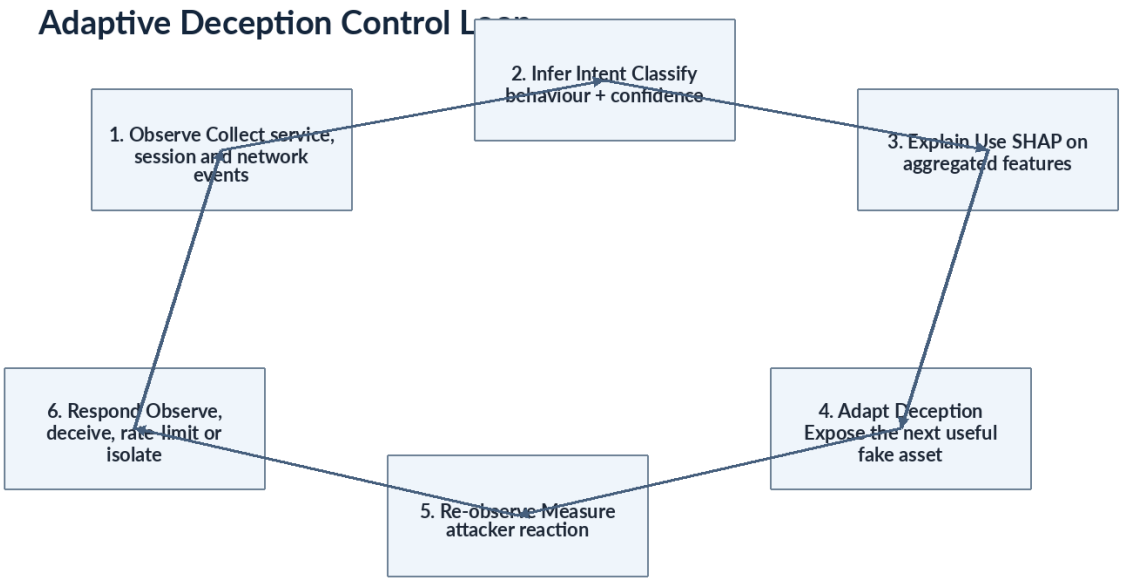
3.2 Design principle

Core principle

The attacker should see only controlled, synthetic assets. Every action is observable, reversible and bounded to the lab environment.

4. System Idea in Plain English

1. A Raspberry Pi exposes a small fake network surface: for example an SSH service and a fake web/API service.
2. An attacker or scripted traffic generator interacts with those services inside the authorized lab.
3. PiSentinel records what happened: connections, ports, failed logins, commands, URLs, file probes, database-like requests, session duration and service transitions.
4. Those events are aggregated into a session profile.
5. A lightweight model and/or rules classify the current behaviour: scanning, reconnaissance, credential attack, remote access, file discovery, data discovery, etc.
6. SHAP explains the prediction using the aggregated features.
7. The deception engine chooses the next fake asset or content to expose. Example: after SSH enumeration, reveal a synthetic admin-like directory tree; after web/API probing, expose additional fake endpoints.
8. Evidence is stored as a hash chain so that later modification of an event can be detected.
9. The forensic layer converts the event stream into an ordered timeline and attack graph.
10. A laptop-hosted or API-hosted AI investigator consumes verified structured evidence and writes an incident explanation, not raw unrestricted system access.
11. A policy engine chooses the next bounded action and the loop continues.



5. Threat Model and Lab Boundaries

5.1 What PiSentinel is designed to observe

- Port scanning and service discovery
- Repeated authentication failures
- SSH enumeration and command patterns
- Web/API endpoint enumeration
- Attempts to access synthetic files
- Attempts to query synthetic database-like records
- Transitions between services
- Rapid repeated requests / automation patterns
- Session duration and interaction depth

5.2 What it must not expose

- Real credentials or secrets
- Real personal data
- Production databases
- Unrestricted shell execution by the AI agent
- Real infrastructure isolation commands outside the test network
- An uncontrolled internet-facing honeypot without containment

Lab rule

All attacker traffic should target synthetic services in a dedicated test network. Blocking/isolation actions should be simulated or restricted to known lab addresses unless you have an explicit, authorized environment where the action is safe.

6. Final System Scope

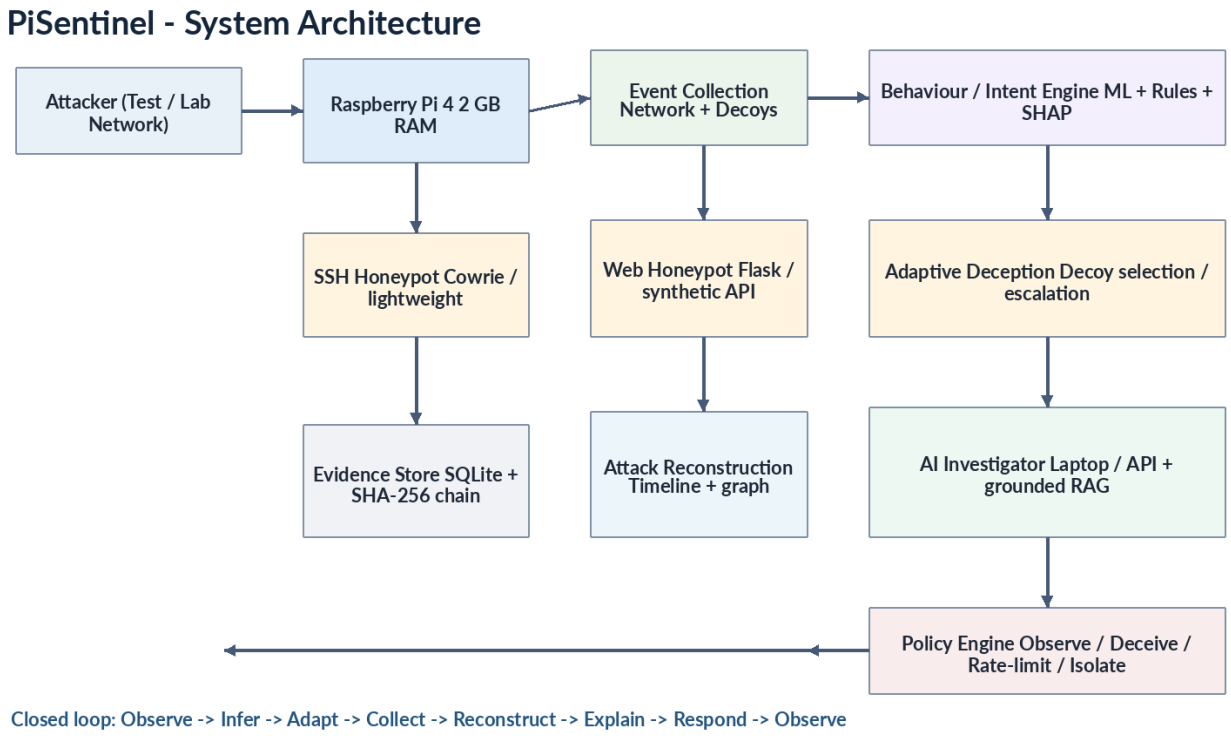
6.1 Minimum viable project

Component	MVP requirement	Optional later extension
Pi collector	Yes	Zeek/Suricata integration
SSH honeypot	Yes	More realistic decoy profiles
Web honeypot	Yes	Multiple synthetic apps/endpoints
SQLite	Yes	PostgreSQL offload
Lightweight ML	Yes	Uncertainty-aware model
SHAP	Yes, on session aggregates	Interactive explanation history
Adaptive deception	Rule-based	Learned policy / contextual bandit as research extension
Hash-chained evidence	Yes	Signed export bundles
Timeline	Yes	Graph analytics / ATT&CK mapping
AI investigator	Laptop/API	RAG over larger case files
Dashboard	Yes	Prometheus/Grafana on laptop

6.2 What is explicitly out of scope for the first version

- Training deep neural networks on the Pi
- Running a large local LLM on the Pi 4
- Building a full SIEM
- Deploying many honeypots at once
- Reinforcement learning from day one
- Blockchain-based evidence storage
- Production-grade enterprise orchestration

7. Architecture



7.1 Data/control flow

```
Attacker -> Decoys/Services -> Event Collector -> Session Builder -> Behaviour Classifier -> SHAP Explanation -> Intent/Confidence -> Adaptive Deception -> Evidence Chain -> Timeline/Attack Graph -> AI Investigator -> Policy Engine -> Next Action -> Attacker
```

7.2 Logical separation

Plane	Runs where	Responsibility
Edge plane	Pi 4	Collection, honeypots, lightweight inference, evidence chain, local control
Analysis plane	Laptop/API	Model training, heavy SHAP rendering, report generation, graph visualization
Presentation plane	Browser	Dashboard, timeline, explanations, decision trace

8. Module-by-Module Design

8.1 Event Collection Layer

Collects network and application events into a normalized schema. Start simple: Python, sockets/log parsing, and structured JSON/SQLite. Integrate heavier sensors only after the pipeline works.

Input: decoy logs + network events. Output: normalized Event objects.

8.2 SSH Honeypot

Provides a controlled fake remote-login surface. Record username attempts, failed/success-like states, session time, commands and file/path probes. Synthetic responses only.

Primary use: credential attack, remote access, enumeration and post-access behaviour.

8.3 Web/API Honeypot

A small Flask/FastAPI service with fake endpoints, login pages, API routes and synthetic data. Log URL, method, request rate, status pattern and endpoint transitions.

Primary use: automated enumeration, web probing and fake data discovery.

8.4 Session Builder

Groups events by source/session/window into a behavioural record. This is the bridge between raw telemetry and ML.

Output: one feature vector per session/window.

8.5 Behaviour Classifier

Start with Random Forest or another lightweight classical model. Keep features interpretable. Use a small label set and document the dataset generation method.

Output: class probabilities + predicted behaviour + confidence.

8.6 Intent Layer

Maps observed behaviour into an attacker-intent abstraction. Intent can be rule-derived initially and later modelled. Keep “prediction” separate from “fact”.

Output: intent label, confidence, evidence list.

8.7 SHAP Explanation

Runs on aggregate session features, ideally only on suspicious sessions or on-demand. It explains the classifier decision without processing every packet.

Output: top positive/negative feature contributions.

8.8 Adaptive Deception Engine

Selects the next decoy content/service based on behaviour stage and confidence. Use deterministic rules first.

Output: next deception action + reason + expiry/rollback.

8.9 Evidence Store

Stores events with timestamp, metadata, previous hash and current hash. Use SQLite for the first implementation.

Output: verifiable incident record.

8.10 Attack Reconstruction

Orders events, groups them into stages and builds an attack graph. NetworkX is enough for the first version.

Output: timeline + graph nodes/edges.

8.11 AI Investigator

Receives only verified structured evidence. The model creates an incident narrative, highlights evidence, explains predictions and distinguishes facts from inference.

Output: grounded report with citations to event IDs.

8.12 Policy Engine

Applies explicit response rules and safety limits. It should never let the LLM directly execute arbitrary commands.

Output: allowed next action.

8.13 Dashboard

Shows current attacker/session, inferred intent, confidence, SHAP features, deception state, timeline, integrity status and response decision.

Output: live operator view.

9. Data Model and Event Schema

Use a stable internal schema even if individual tools emit different log formats. This makes the project easier to test, explain and demonstrate.

Field	Example	Purpose
event_id	EVT-000231	Stable identifier for evidence references
timestamp	2026-09-16T14:22:01+05:30	Ordering / timeline
source_ip	10.10.0.20	Attacker/session attribution inside the lab
destination_service	ssh	Service targeted
event_type	auth_failure	Normalized event category
action	login_attempt	What happened
metadata	{"username":"admin"}	Structured contextual data
session_id	S-014	Grouping events into behaviour windows
severity	medium	Risk policy input
previous_hash	abc...	Chain linkage
current_hash	def...	Integrity proof
model_label	credential_attack	ML result, not ground truth
confidence	0.91	Model certainty
shap_summary	failed_logins + unique_ports	Explanation
response_action	increase_deception	Policy result

9.1 Example event JSON

```
{
  "event_id": "EVT-000231",
  "timestamp": "2026-09-16T14:22:01+05:30",
  "source_ip": "10.10.0.20",
  "destination_service": "ssh",
  "event_type": "auth_failure",
  "action": "login_attempt",
  "metadata": {"username": "admin"},
  "session_id": "S-014",
  "severity": "medium"
}
```

10. Behaviour Features and Intent Classification

10.1 Feature vector

Feature family	Examples	Why it matters
Network	connection_count, unique_ports, unique_services	Separates broad recon from focused service interaction
Authentication	failed_login_count, unique_usernames	Supports credential-attack inference
Command/session	command_count, session_duration	Shows depth of interaction
Web	http_request_count, unique_urls, endpoint_count	Supports enumeration / application probing
Data access	file_access_count, db_like_request_count	Shows movement toward discovery
Sequence	service_transition_count, repeat_pattern_score	Captures progression rather than a single event

10.2 Suggested labels

- normal / benign test traffic
- reconnaissance
- automated scanning
- credential attack
- remote access
- file discovery
- data discovery

10.3 First model

A Random Forest is a sensible baseline because it handles mixed numeric features, has fast inference, is easy to explain with Tree SHAP, and is practical on a small Pi. The first objective is a reproducible pipeline, not maximum model complexity.

```
RandomForestClassifier(
    n_estimators=50,
    max_depth=8,
    n_jobs=1,
    random_state=42
)
```

10.4 Dataset strategy

The project should build its own small labelled lab dataset using scripted attack scenarios and benign interactions. Each scenario should have a known start/end time and ground-truth stage sequence. This gives you a defensible evaluation set without using real user data.

11. SHAP Explainability

SHAP is not the classifier. It is an explanation layer over the classifier. For PiSentinel, the most useful unit of explanation is a session or sliding behavioural window rather than every network packet.

11.1 Example

Example explanation

Prediction: credential_attack (0.91 confidence). Positive contributors: failed_login_count, unique_usernames,

request_rate. Negative contributor: short_session_duration. Interpretation: the model is being pushed primarily by repeated authentication attempts across multiple usernames.

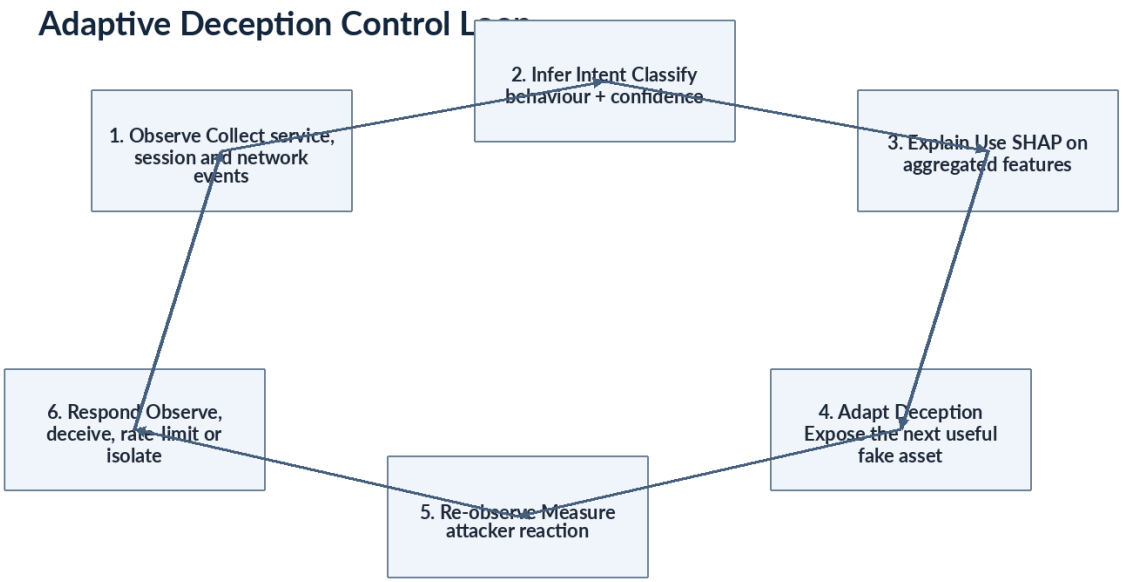
11.2 Pipeline

```
Session events -> Feature vector -> Random Forest -> class probability -> SHAP TreeExplainer -> top contributors -> dashboard / evidence record
```

11.3 What SHAP should and should not do

- Should: explain model behaviour in human-readable terms.
- Should: preserve a machine-readable list of features and contribution values.
- Should not: be treated as proof of attacker intent.
- Should not: be generated continuously for every packet on a 2 GB Pi.
- Should: be stored with the specific model/version and feature vector used.

12. Adaptive Cyber Deception



12.1 Deception state machine

Observed state	Example trigger	Deception action	Purpose
Recon	Many ports / services touched quickly	Expose a small synthetic service inventory	Encourage deeper interaction
SSH enumeration	Repeated username/path probes	Expose synthetic user names and directory structure	Move attacker toward controlled discovery
Credential attack	High failed-login rate	Present a synthetic admin-like account surface	Capture behaviour without real secrets
Web enumeration	Many URLs / methods / endpoints	Reveal additional fake API routes	Measure next-stage behaviour
Data discovery	File/DB-like probes after access	Expose synthetic records	Collect interaction depth
High risk	Repeated multi-stage activity +	Switch to containment policy in	Limit spread and preserve state

Observed state	Example trigger	Deception action	Purpose
	high confidence	lab	

12.2 Why rule-based first?

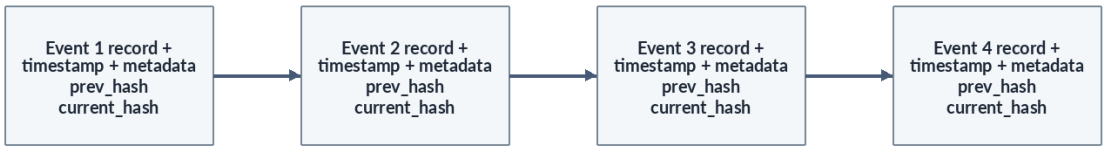
Rule-based adaptation is easier to debug, safer to constrain and easier to explain in a final-year project. A learned policy can be a later research extension after the deterministic loop works and the project has data.

12.3 Pseudocode

```
if intent == "reconnaissance":
    action = EXPOSE_CONTROLLED_SERVICE
elif intent == "credential_attack":
    action = EXPOSE_SYNTHETIC_IDENTITY_SURFACE
elif intent == "file_discovery":
    action = EXPOSE_SYNTHETIC_FILE_TREE
elif intent == "data_discovery":
    action = EXPOSE_SYNTHETIC_DATASET
elif risk >= HIGH_THRESHOLD:
    action = ISOLATE_LAB_SESSION
else:
    action = OBSERVE
record_decision(reason, confidence, evidence_ids)
```

13. Digital Forensics and Evidence Integrity

Tamper-Evident Evidence Chain



current_hash = SHA-256(event_data + previous_hash)

13.1 Hash-chain design

Each evidence record stores the hash of the previous record. The current hash is calculated over the event content plus the previous hash. This does not make the evidence magically “legal”; it gives the project a simple, auditable mechanism for detecting post-hoc modification.

```
current_hash = SHA256(canonical_event_json + previous_hash)
```

13.2 Integrity verification

12. Recompute the hash of every event in order.
13. Compare it to the stored current_hash.
14. Confirm the stored previous_hash matches the previous event current_hash.
15. Report the first broken link and the affected event IDs.

13.3 Evidence bundle

Artifact	Purpose
events.jsonl	Raw normalized event records
case.json	Case metadata, source/session scope, model versions
timeline.json	Ordered stage reconstruction
attack_graph.json	Nodes/edges and evidence references
integrity_report.json	Hash verification result
shap.json	Model explanation for selected sessions
investigator_report.md	Human-readable incident report

14. Attack Timeline and Graph Reconstruction

The timeline is intentionally simple: order events, collapse low-level noise into stages, and attach evidence IDs to every stage. The graph adds relationships such as “source interacted with service”, “service led to file discovery”, and “event followed event”.

14.1 Example timeline

Time	Stage	Observed event	Evidence
t0	Recon	Multiple TCP connections / ports	EVT-01..EVT-08
t1	SSH enumeration	Repeated usernames and paths	EVT-09..EVT-17
t2	Credential attack	High failed-login rate	EVT-18..EVT-29
t3	Remote access	Synthetic session accepted	EVT-30
t4	File discovery	Directory/path probes	EVT-31..EVT-41
t5	Data discovery	DB-like requests	EVT-42..EVT-50

14.2 Graph model

```
Attacker -> Service -> Session -> Action -> Resource -> Evidence
Example: 10.10.0.20 -> SSH -> S14 -> list_directory -> /var/www_fake -> EVT-31
```

15. Evidence-Grounded AI Investigator

15.1 Role

The AI investigator is a reporting and reasoning assistant. It should synthesize verified evidence, explain the model output, summarize the timeline, highlight uncertainty and generate next-step recommendations. It should not be the component that directly controls the operating system.

15.2 Input contract

```
{
  "case_id": "CASE-004",
  "integrity": {"status": "verified"},
  "timeline": [...],
  "attack_graph": {...},
  "intent": {"label": "credential_attack", "confidence": 0.91},
  "shap": {"top_features": [...]},
```

```
  "evidence": ["EVT-18", "EVT-19", "EVT-20"]
}
```

15.3 Required report structure

- Observed facts - only what evidence shows.
- Model assessment - what the ML/rules predicted.
- Why the model predicted it - SHAP summary.
- Attack progression - timeline and graph narrative.
- Uncertainty / missing evidence - what is not known.
- Recommended bounded response - from the policy engine, not direct free-form commands.

15.4 Grounding guardrails

- Every factual statement in the generated report should reference one or more evidence IDs.
- Predictions must be labelled as predictions.
- If evidence is missing, state that rather than inventing a detail.
- LLM output should be stored separately from raw evidence.
- Human-readable recommendations should not bypass the policy engine.

16. Risk-Aware Response Engine

Risk level	Confidence / context	Allowed action	Why
Low	Weak evidence or ambiguous behaviour	Observe	Avoid premature escalation
Medium	Suspicious repeated activity	Increase deception	Collect more evidence
High	Multi-stage behaviour with strong evidence	Rate-limit / restrict lab session	Reduce uncontrolled interaction
Critical	Confirmed high-risk path in lab	Isolate / stop test session	Contain safely while preserving evidence

16.1 Important separation

Never let the LLM become the root user

The AI can recommend or select from a finite action vocabulary. The policy engine validates the action. The executor performs only pre-approved lab-safe operations.

17. Implementation Stack

Layer	Recommended technology	Notes
OS	Raspberry Pi OS 64-bit	Keep base image minimal
Language	Python	Fast iteration and good security tooling ecosystem
API	FastAPI	Lightweight control/data API
SSH decoy	Cowrie or custom minimal SSH decoy	Start with one service
Web decoy	Flask or FastAPI	Synthetic endpoints and logs
Database	SQLite	Best fit for MVP on 2 GB RAM
ML	scikit-learn	Random Forest baseline

Layer	Recommended technology	Notes
Explainability	SHAP	TreeExplainer for RF
Graph	NetworkX	Timeline-to-graph conversion
Dashboard	Simple React / HTML/JS	Avoid heavy monitoring stack on Pi
Containerization	Docker, selectively	Only after native prototype works
AI	Hosted/API LLM on laptop or trusted service	Grounded JSON input only
Version control	Git + GitHub	Track model/data/code versions

18. Raspberry Pi 4 Resource Strategy

Because the Pi has 2 GB RAM, performance discipline is part of the research story. The goal is not to cram every component onto the device. The goal is to prove a useful closed loop under resource constraints.

Component	Target approach	Approx. memory budget / behaviour
OS	64-bit minimal setup	~300-500 MB
Collector/API	Single Python process where possible	~50-150 MB
SQLite	Local file database	~20-100 MB depending on case size
SSH honeypot	One instance	~100-250 MB
Web honeypot	One lightweight app	~30-100 MB
ML inference	Small RF, one process	~50-150 MB
SHAP	On-demand only	~100-300 MB transient
Dashboard	Simple local server	~50-150 MB
Heavy analysis	Run on laptop/API	Offloaded

Resource rule

Do not run every optional service at once. Build the native MVP first, then containerize only the components that improve reproducibility without exhausting RAM.

19. Build Plan - Week by Week

Week 1 - Foundation

Flash/configure Pi; static IP; SSH administration; project repository; basic FastAPI; SQLite schema; health endpoint; dashboard shell.

Milestone: Pi boots and dashboard shows system status.

Week 2 - Event collection + decoys

Implement SSH decoy; implement web/API decoy; normalize logs; event IDs; session IDs.

Milestone: Attacker interaction produces structured events.

Week 3 - Dataset + baseline ML

Write scenario generators; collect benign and malicious lab traces; feature extractor; train/test split; Random Forest baseline.

Milestone: Model predicts basic behaviour classes.

Week 4 - SHAP + intent

Integrate Tree SHAP; top-feature explanations; confidence; rule-to-intent mapping; store explanations.

Milestone: Dashboard explains a suspicious session.

Week 5 - Adaptive deception

Implement deterministic state machine; fake users/files/endpoints; action expiry; rollback.

Milestone: The visible decoy changes based on behaviour.

Week 6 - Evidence + reconstruction

Hash chaining; integrity verifier; timeline builder; NetworkX graph.

Milestone: A case can be exported and independently verified.

Week 7 - AI investigator

Structured evidence package; LLM prompt/schema; evidence references; report generation; uncertainty.

Milestone: Incident report is generated from a completed case.

Week 8 - Policy engine + integration

Observe/deceive/rate-limit/isolate logic; action audit log; end-to-end integration.

Milestone: Full closed loop works in the lab.

Weeks 9-10 - Hardening

Resource optimization; failure handling; dataset cleanup; repeatable attack scripts; UI polish; install script.

Milestone: Stable demo build with repeatable results.

Weeks 11-12 - Evaluation + report

Run experiments; collect metrics; make graphs/tables; write chapters; prepare viva and presentation.

Milestone: Research-ready final package.

20. Experimental Methodology

20.1 Baseline comparisons

- Static deception vs adaptive deception
- ML intent classification with vs without behavioural features
- ML decision with vs without SHAP explanation layer
- Rule-only intent vs ML-assisted intent
- Timeline reconstruction from raw events vs reconstructed stages
- Local-only processing vs Pi-plus-laptop offload

20.2 Experimental protocol

16. Use a fixed set of scripted attack scenarios.
17. Repeat each scenario multiple times.
18. Keep hardware and software versions fixed per experiment.
19. Log the exact model version and feature schema.
20. Measure both cybersecurity outcomes and edge-resource costs.
21. Report failure cases, not only successful demonstrations.

21. Metrics and Evaluation

Metric	Definition / example	Collection method
Intent accuracy	Correct predicted intent / total labelled sessions	Ground-truth scenario labels
Precision / Recall / F1	Per-class and macro metrics	scikit-learn
Attack-stage accuracy	Correctly reconstructed stages / labelled stages	Scenario ground truth
Timeline accuracy	Correct order / temporal alignment	Event timestamps vs script
Interaction time	Time attacker remains engaged with decoys	Case session timestamps
Interaction depth	Number of meaningful decoy interactions	Event classification
Decoys reached	Count of adaptive assets touched	Deception logs
Integrity failure detection	Injected modifications detected	Synthetic tampering tests
Grounding rate	Report statements linked to evidence IDs	Post-process validator
CPU usage	Average / peak Pi CPU	psutil
RAM usage	Average / peak resident memory	psutil
Inference latency	Feature-to-decision time	Monotonic timestamps
Network overhead	Bytes sent to analysis plane	System/network counters

22. Test Cases and Demo Scenarios

Scenario	Attack pattern	Expected PiSentinel behaviour	Primary evidence
Scenario A - Port scan	Scanner touches many ports/services.	Recon detected; deception remains controlled.	Ports, timing, service count.
Scenario B - SSH credential attack	Repeated usernames/password attempts.	Credential-attack intent; synthetic identity surface exposed.	Failures, usernames, rate.
Scenario C - Post-access discovery	Synthetic shell-like interaction then file/path probes.	Remote-access + file-discovery stages reconstructed.	Commands, paths, session depth.
Scenario D - Web enumeration	Many URLs, methods and fake API endpoints.	Web deception expands; attacker behaviour captured.	Unique URLs, request rate.
Scenario E - Multi-stage	Recon -> SSH -> discovery -> DB-like requests.	Timeline + graph show stage progression; report explains.	Stage order, evidence references.
Scenario F - Evidence tampering	Modify one stored event after collection.	Integrity verifier flags broken chain.	Broken hash link / event ID.

22.1 Final demo narrative

22. Start PiSentinel and show green health status.
23. Launch a controlled scan from a lab machine.

24. Show the session appear on the dashboard.
25. Switch to SSH enumeration and repeated login attempts.
26. Show the intent confidence increase and SHAP features.
27. Trigger adaptive exposure of synthetic users/files.
28. Continue into file/data discovery.
29. Open the attack timeline and graph.
30. Open the evidence integrity panel.
31. Generate the grounded AI incident report.
32. Show the policy decision and audit trail.
33. Modify one evidence row in a copy and run verification to demonstrate tamper detection.

23. Security, Safety and Isolation

- Run the entire project inside a private lab subnet/VLAN.
- Bind synthetic services to controlled interfaces/IPs.
- Use fake credentials and synthetic records only.
- Do not allow the AI layer to execute arbitrary shell commands.
- Use an explicit action allowlist.
- Add rate limits to decoys so a test script cannot exhaust the Pi.
- Log every automated control decision.
- Keep rollback controls for deception changes.
- Prefer simulated isolation during evaluation unless the lab setup is explicitly authorized for live isolation.
- Back up case evidence before performing destructive test actions.

24. Failure Modes and Fallbacks

Failure	Likely cause	Fallback
Model uncertain	Novel or mixed behaviour	Switch to observe + gather more evidence
SHAP too slow	Large feature vector / too frequent calls	Run only on suspicious sessions, reduce features
Pi RAM pressure	Too many services/processes	Disable optional dashboard/container, offload analysis
Decoy loop oscillates	Rules trigger repeatedly	Add cooldown timers and state transitions
LLM hallucinates	Weak evidence grounding	Require evidence IDs; reject ungrounded fields
Hash chain breaks	Storage modification / bug	Flag case as integrity-failed; preserve raw copy
Timeline misorders events	Clock skew or inconsistent timestamps	Use monotonic sequence IDs + timestamp normalization
AI unavailable	API/network outage	Generate deterministic template report from verified evidence

25. What Not to Build

- Do not begin with 8-10 honeypot services. One SSH + one web decoy is enough for the first research loop.
- Do not begin with reinforcement learning. A deterministic adaptive policy provides a baseline that is easier to test.
- Do not build a blockchain just because the word “forensics” appears in the title. Hash chaining is simpler and sufficient for the integrity experiment.
- Do not run a large language model locally on the Pi 4 2 GB for the first version.

- Do not make the dashboard the project. The research value is in the closed control loop and the evidence trail.
- Do not train a complex model before you have a clean dataset and repeatable attack scenarios.
- Do not claim the system proves attacker intent. It estimates intent from observed behaviour and should expose confidence/uncertainty.

26. Final Deliverables

Deliverable	Contents
Hardware demo	Raspberry Pi 4 + controlled lab network + fake services
Source code	Collector, decoys, feature extractor, ML, SHAP, deception, forensics, policy engine, dashboard
Dataset	Benign + scripted attack sessions with labels and scenario metadata
Model artifacts	Trained model + version + feature schema
Evidence format	JSON/JSONL export + integrity verifier
Attack reconstruction	Timeline + graph JSON / visualization
AI report	Grounded incident summary with evidence references
Evaluation	Metrics, comparison experiments, resource measurements
Documentation	Architecture, setup, API, threat model, limitations
Final presentation	Problem -> idea -> architecture -> demo -> results -> contribution

27. Viva / Evaluation Questions

Question	Strong answer
Why is this not just a honeypot?	Because deception is only one module. The system additionally classifies behaviour, explains decisions, preserves tamper-evident evidence, reconstructs the attack and uses an evidence-grounded investigator.
Why Raspberry Pi 4?	It creates a genuine resource-constrained edge scenario and forces explicit trade-offs between local and offloaded computation.
Why Random Forest first?	It is lightweight, fast and interpretable enough for a strong baseline and works naturally with Tree SHAP.
What does SHAP explain?	The model decision over the aggregated session feature vector - not the attacker's actual mental intent.
How is deception adaptive?	A state machine maps observed behaviour and confidence to a finite set of synthetic assets or response actions.
How do you prove evidence was not changed?	Recompute the SHA-256 chain and verify every previous/current hash link.
Why not blockchain?	The experiment needs tamper detection and provenance, not decentralized consensus.
What if the AI is wrong?	The AI cannot directly execute arbitrary actions. The policy engine limits action choices, and the report distinguishes observed facts, model predictions and recommendations.
What is your research variable?	The main independent comparison is static versus adaptive deception, with attacker engagement, reconstruction quality and resource cost as outcomes.

Question	Strong answer
What is a limitation?	The lab uses synthetic attacks and services, so real-world generalization must be treated cautiously.

28. Glossary

Term	Meaning in this project
Adaptive deception	Changing the fake environment based on observed attacker behaviour.
Behavioural feature	A numerical summary of a session such as failed logins or unique ports.
Intent	A high-level inferred goal such as credential attack or data discovery.
SHAP	A method for explaining feature contributions to a model prediction.
Tamper-evident	Designed to reveal modification through integrity checks.
Evidence chain	Ordered records linked by cryptographic hashes.
Attack graph	Graph representation of attacker, services, actions and resources.
Grounded AI	AI output constrained by a supplied evidence set.
Policy engine	Finite, auditable rules that approve or reject defensive actions.
Decoy	Synthetic service, account, file, endpoint or dataset used to engage/observe an attacker.

29. Final Master Checklist

- Pi 4 configured and reproducible
- Private lab network documented
- SSH decoy works
- Web decoy works
- Events normalized into one schema
- Session builder works
- Dataset labelled
- Baseline ML measured
- SHAP stored for suspicious sessions
- Intent confidence displayed
- Adaptive deception rules implemented with cooldowns
- Hash chain implemented and verified
- Timeline generated
- Attack graph generated
- Grounded AI report generated
- Policy engine prevents arbitrary AI commands
- Dashboard shows end-to-end case
- Resource usage measured
- Static vs adaptive experiment completed
- Failure cases documented
- Final demo script repeated successfully
- Source code tagged and backed up

- Final report and presentation prepared

The project in one sentence

PiSentinel is a resource-aware cyber-defense loop for the edge: it observes attacker behaviour, infers intent with explainable lightweight ML, adapts controlled deception, preserves tamper-evident evidence, reconstructs the attack, and uses grounded AI to explain and support bounded response decisions.

Appendix A - Suggested Repository Structure

```
pisentinel/
├── edge/
│   ├── collector/
│   ├── decoys/
│   │   ├── ssh/
│   │   └── web/
│   ├── features/
│   ├── model/
│   ├── explain/
│   ├── deception/
│   ├── evidence/
│   ├── reconstruction/
│   ├── policy/
│   └── api/
├── analysis/
│   ├── training/
│   ├── shap_reports/
│   ├── graphs/
│   └── investigator/
├── dashboard/
├── scenarios/
├── data/
│   ├── raw/
│   ├── processed/
│   └── cases/
├── docs/
└── scripts/
```

Appendix B - Suggested API Endpoints

Endpoint	Method	Purpose
/health	GET	System health
/events	GET	Recent normalized events
/sessions	GET	Session summaries
/sessions/{id}	GET	Session + features + intent
/explanations/{id}	GET	SHAP explanation
/cases/{id}	GET	Case summary
/cases/{id}/timeline	GET	Timeline
/cases/{id}/integrity	GET	Integrity verification
/decoy/state	GET	Current deception state
/policy/decision	GET	Latest policy decision
/reports/{id}	GET	Grounded investigator report

Appendix C - First Coding Order

34. 1. config.py + .env.example
35. 2. SQLite schema + Event model
36. 3. event writer + hash chain
37. 4. SSH decoy
38. 5. web decoy

39. 6. session builder
40. 7. feature extractor
41. 8. dataset scripts
42. 9. Random Forest training
43. 10. SHAP service
44. 11. intent mapper
45. 12. deception state machine
46. 13. timeline builder
47. 14. graph builder
48. 15. policy engine
49. 16. API endpoints
50. 17. dashboard
51. 18. AI investigator adapter
52. 19. integration tests
53. 20. evaluation scripts